МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
«НСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ»

ЛАБОРАТОРНА РОБОТА №5

з курсу: «Системи баз даних»

Виконав:
студент III курсу
групи ДА-11
Сидько Нікіта

КИЇВ – 2023

Системи баз даних «ключ / значення»

Redis - це відкрита база даних типу ключ-значення з відкритим вихідним кодом. Redis зберігає дані в оперативній пам'яті, що забезпечує високу швидкість доступу до даних. Redis також підтримує ряд структур даних, включаючи списки, множини, хеші та зсуви.

Встановлення сбд Redis

Для встановлення цієї СБД, як контейнера у Docker пропишемо в консолі наступне :

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)

PS C:\Users\IVA-PC> docker run --name some-redis -p 6379:6379 -d redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
360eba32fa65: Already exists
64b323c8825b: Pull complete
383536099eae: Pull complete
e3f0c12dcc8e: Pull complete
2d2142a35b4d: Pull complete
b4cada01e6a7: Pull complete
Digest: sha256:f92a0be0ba8c085e6a5e2d2bea386365443485bcd67ced5ca8ddcdacdd4656d2
Status: Downloaded newer image for redis:latest
218023fe6190af7b837ad43b5759de36db8f41589f1c90bd2bce28d45fb2bb82
PS C:\Users\IVA-PC>
```

Після чого, можна побачити, що в нас з’явився новий контенер **some-redis**

	some-couchdb 6fa1a08d1960	couchdb	Exited	0%	5984:5984	3 hours ago
	some-hbase 10e27fcb7f20	harisekhon/hbase	Exited (137)	0%		25 days ago
	some-redis 86902c21c8c9	redis	Running	0%	6379:6379	8 minutes ago

Створення БД

Виконуємо запуск консолі СБД, яка автоматично під’єднається до сервера в локальній мережі за портом 6379. У відповідь на команду help СБД поверне інструкції щодо застосування довідки. При зазначенні перших літер і переміщенні на клавіатурі клавішею Tab СБД показує написання наявних команд

```
# redis-cli
127.0.0.1:6379> help
redis-cli 7.2.1
To get help about Redis commands type:
      "help @<group>" to get a list of commands in <group>
      "help <command>" for help on <command>
      "help <tab>" to get a list of possible help topics
      "quit" to exit

To set redis-cli preferences:
      ":set hints" enable online hints
      ":set nohints" disable online hints
Set your preferences in ~/.redisclirc
127.0.0.1:6379>
```

Redis підтримує кілька опцій дублювання бази даних. Стандартним є зберігання тільки у внутрішній пам’яті, наприклад оперативній. Команда **LASTSAVE** зчитує у форматі Unix значення змінної часу, що відповідає часовій позначці завершального дублювання вмісту БД в зовнішню пам’ять.

Команда **SAVE** (або BGSAVE) забезпечує увімкнення асинхронного фонового дублювання даних у зовнішню пам’ять.

```
127.0.0.1:6379> LASTSAVE
(integer) 1700402673
127.0.0.1:6379> SAVE
OK
```

Також тут є тимчасове зберігання екземплярів даних, щоб уникати надмірного розростання БД, для застосування маємо такі команди:

- **EXPIRE** - встановлює інтервал зберігання, параметрами є ключ екземпляру даних і цілочисельне значення у секундах,
- **EXISTS -** перевіряє статус екземпляру

```
127.0.0.1:6379> SET bob "I'm fly"
OK
127.0.0.1:6379> EXPIRE bob 10
(integer) 1
127.0.0.1:6379> time
1) "1700404095"
2) "74215"
127.0.0.1:6379> EXISTS bob
(integer) 1
127.0.0.1:6379> time
1) "1700404108"
2) "403852"
127.0.0.1:6379> time
1) "1700404111"
2) "72637"
127.0.0.1:6379> EXISTS bob
(integer) 0
127.0.0.1:6379>
```

У Redis реалізовано команду **SETEX**, щоб об’єднати створення екземпляра й встановлення інтервалу зберігання. Команда **TTL** (time to live) зчитує різницю часу до видалення екземпляра даних.

```
127.0.0.1:6379> SETEX ice 10 "I'm melting..."
OK
127.0.0.1:6379> TTL ice
(integer) 8
127.0.0.1:6379> TTL ice
(integer) 4
127.0.0.1:6379> TTL ice
(integer) 1
127.0.0.1:6379> EXISTS ice
(integer) 0
```

будь-який момент до видалення тимчасовість зберігання можна скасувати командою *PERSIST*.

```
127.0.0.1:6379> PERSIST ice
(integer) 0
127.0.0.1:6379>
```

CRUD-запити до бази даних

Команда **SET** зіставляє скорочений ключ, наприклад gog, зі значенням повного написання вебресурсу google.com . Для зчитування значення за ключем виконуємо команду **GET**.

```
127.0.0.1:6379> SET gog http://google.com
OK
127.0.0.1:6379> GET gog
"http://google.com"
```

Для зменшення кількості запитів реалізовано команду **MSET**, яка створює кілька екземплярів «ключ / значення». Зіставимо Youtube.com зі значенням yt, а twitch.tv — з tv. Далі зчитаємо командою **MGET** значення за списком ключів.

```
127.0.0.1:6379> SET gog http://google.com
OK
127.0.0.1:6379> GET gog
"http://google.com"
127.0.0.1:6379> MSET yt youtube.com tw twitch.tv
OK
127.0.0.1:6379> MGET gog yt tw
1) "http://google.com"
2) "youtube.com"
3) "twitch.tv"
127.0.0.1:6379>
```

Крім строкових, СБД розрізняє і цілочисельні значення. У Redis реалізовано команди обчислення статистики за екземплярами даних. **INCR** визначає, що значення екземпляра цілочисельне й збільшує значення на одиницю, в свою чергу зменшення виконується командою **DECR**

```
127.0.0.1:6379> SET counter 3
OK
127.0.0.1:6379> INCR counter
(integer) 4
127.0.0.1:6379> GET counter
"4"
127.0.0.1:6379> DECR counter
(integer) 3
127.0.0.1:6379> GET counter
"3"
```

При збільшенні строкового значення на одиницю СБД поверне помилку

```
127.0.0.1:6379> SET bad_count "a"
OK
127.0.0.1:6379> INCR bad_count
(error) ERR value is not an integer or out of range
```

Простір наймнувань

Простір найменувань в СБД Redis — база даних. Стандартно всі команди виконуються в просторі найменувань № 0.

Зробимо запис та зчитування з простору найменувань № 0

```
127.0.0.1:6379> SET student Bob
OK
127.0.0.1:6379> GET student
"Bob"
```

Якщо командою **SELECT** переміститись у простір № 1, виявиться, що екземпляр даних **student** не створено

```
127.0.0.1:6379[1]> GET student
(nil)
```

Зробимо дозапис даних у простір № 1

```
127.0.0.1:6379[1]> SET student Mike
OK
127.0.0.1:6379[1]> GET student
"Mike"
127.0.0.1:6379[1]>
```

Перейдемо до простору 0 використовуючи команду **SELECT** 0

```
127.0.0.1:6379[1]> SELECT 0
OK
127.0.0.1:6379> GET student
"Bob"
```

Для переміщення кортежів між просторами найменувань реалізовано команду **MOVE**. Перемістимо екземпляр даних **student** з простору 0 у простір № 2, зчитаємо його та повернемось до простору 0

```
127.0.0.1:6379> MOVE student 2
(integer) 1
127.0.0.1:6379> SELECT 2
OK
127.0.0.1:6379[2]> GET student
"Bob"
127.0.0.1:6379[2]> SELECT 0
OK
127.0.0.1:6379>
```

Структури даних

СБД Redis підтримує зберігання до 2^32 значень у вигляді списку з єдиним ключем. Найменування команд у Redis характеризуються структурами даних. Команди для множин починаються з літери S (set), для хешів — з H (hash), для відсортованих множин — із Z.Команди для списків починаються з літер L (left — зліва) чи R (right — справа) відповідно до направленості виконання запитів

Хеш

Хеші виконують у Redis функцію об’єктів-контейнерів, здатних зберігати довільну кількість кортежів «ключ / значення». Застосуємо хеш для статистики щодо користувачів, які навчаються в ВНЗ. Дозапишемо в хеш ієрархічне об’єднання ключів і значень, сегментованих позначкою «:» за їхньою вкладеністю.

```
127.0.0.1:6379> MSET studentID:bob:studentname "Bob Woody" studentID:bob:password 123
OK
127.0.0.1:6379> MGET studentID:bob:studentname studentID:bob:password
1) "Bob Woody"
2) "123"
```

Команда **HVALS** зчитує значення кортежів хешу за строковим ідентифікатором, а команда **HKEYS** зчитує ключі кортежів хешу:

```
127.0.0.1:6379> HMSET studentID:bob name "Bob Woody" password 123
OK
127.0.0.1:6379> HKEYS studentID:bob
1) "name"
2) "password"
127.0.0.1:6379> HVALS studentID:bob
1) "Bob Woody"
2) "123"
127.0.0.1:6379>
```

Для зчитування значення за ключем кортежу зазначаються ідентифікатор хешу (ідентифікатор множини кортежів) і ключ конкретного кортежу

```
127.0.0.1:6379> HGET studentID:bob password
"123"
```

Команда **HDEL** — видалення значення з хешу, **HINCRBY** — збільшення на одиницю, **HLEN** — розрахунок загальної кількості значень у хеші.

Список

Список містить послідовність значень і застосовується як черга (перший прийшов — перший оброблюється), як і стек (останній прийшов — перший оброблюється). Підтримується дозаписування в середину списку й переміщення значень між списками.

Реалізуємо зберігання списку URL, які збираються відвідати студенти для виконання лабораторної роботи.

Команда **LLEN** зчитує кількість значень, наявних у списку. Команда **LRANGE** зчитує частину списку за вказаним інтервалом номерів послідовності. Перше значення має індекс 0. Від’ємне значення позначає початок відліку із закінчення списку.

```
127.0.0.1:6379> RPUSH bob:wishlist gog tw yt
(integer) 3
127.0.0.1:6379> LRANGE bob:wishlist 0 -1
1) "gog"
2) "tw"
3) "yt"
```

Якщо кількість дорівнює 0, команда LREM видаляє всі знайдені кортежі з вказаним значенням ключа. Якщо лічильник більший за 0, вказану кількість значень буде видалено.

```
127.0.0.1:6379> LREM bob:wishlist 0 gog
(integer) 1
```

Для застосування списку як стеку, потрібно дозаписувати значення командою **RPUSH** і зчитувати — **RPOP**.

Для переміщення значень між списками в Redis реалізовано команду **RPOPLPUSH** — об’єднання команд видалення з кінця поточного списку (pop справа) і дозаписування на початку цільового списку (push зліва).

```
127.0.0.1:6379> RPOPLPUSH bob:wishlist bob:visited
"yt"
```

Множина

Доповнимо сервіс опцією об’єднання URL за категоріями, наприклад «математика», «англійська» тощо.

Множиною називається список, який не містить повторюваних значень і призначений обчислювати об’єднання, перетин або різницю. Для об’єднання URL за загальним ключем створюють множину й дозаписують кілька значень командою **SADD**:

```
127.0.0.1:6379> SADD math kpimath.com onlinemath.com
(integer) 2
```

Команда **SMEMBERS** зчитує всі значення множини, але без сортування.

```
127.0.0.1:6379> SMEMBERS math
1) "kpimath.com"
2) "onlinemath.com"
127.0.0.1:6379>
```

Створимо категорію english для вебресурсів що допоможуть студентам у вивчені англійської мови.

```
127.0.0.1:6379> SADD english redford.com greenforest.com
(integer) 2
```

Щоб з’ясувати, які сайти допоможуть вивчити і математику і англійську виконаємо перетин множин командою **SINTER**:

```
127.0.0.1:6379> SADD math mathonenglish.com
(integer) 1
127.0.0.1:6379> SADD english mathonenglish.com
(integer) 1
127.0.0.1:6379> SINTER math english
1) "mathonenglish.com"
```

Щоб розрахувати різницю множин, виконаємо команду **SDIFF**

```
127.0.0.1:6379> SDIFF english math
1) "redford.com"
2) "greenforest.com"
127.0.0.1:6379> SDIFF math english
1) "kpimath.com"
2) "onlinemath.com"
```

При об’єднанні множин (команда **SUNION**)ідентичні значення видаляються в нашому випадку це буде **mathonenglish**

```
127.0.0.1:6379> SUNION english math
1) "redford.com"
2) "greenforest.com"
3) "mathonenglish.com"
4) "kpimath.com"
5) "onlinemath.com"
```

Команда **SUNIONSTORE** за результатом об’єднання створює множину:

```
127.0.0.1:6379> SUNIONSTORE lessons math english
(integer) 5
127.0.0.1:6379> SMEMBERS lessons
1) "kpimath.com"
2) "onlinemath.com"
3) "mathonenglish.com"
4) "redford.com"
5) "greenforest.com"
```

У Redis для зберігання перетину в новій множині реалізовано команду **SINTERSTORE**, для зберігання різниці — **SDIFFSTORE**. **SMOVE** переміщує значення між множинами; **SREM** key [value ...] видаляє конкретне значення; **SCARD** обчислює загальну кількість значень.

Відсортовані множини

Відсортовані множини структуровані, як списки, і не містять повторюваних значень, як множини. Щоб дозаписати значення у відсортовану множину, треба вказати найменування множини й масив кортежів «ключ / значення», де ключ — цілочисельний лічильник, значення — строкове

```
127.0.0.1:6379> ZADD visits 100 gog 1000 yt 9999 tw
(integer) 3
```

ZINCRBY збільшує лічильник на фіксоване значення (якщо значення від’ємне, то зменшує):

```
127.0.0.1:6379> ZINCRBY visits 1 tw
"10000"
127.0.0.1:6379>
```

Створимо новий ключ **ratings** з оцінками користувачів щодо вебресурсів:

```
127.0.0.1:6379> ZADD ratings 56 gog 76 yt 78 tw
(integer) 3
```

Об’єднаємо значення множин рейтингу і відвідувань вебсайтів, розрахуємо на базі них значення важливості вебресурсів. Збільшимо **ratings** вдвічі. Зчитаємо відсортовану множину в інтервалі значень.

```
127.0.0.1:6379> ZUNIONSTORE importance 2 visits ratings WEIGHTS 1 2 AGGREGATE SUM
(integer) 3
127.0.0.1:6379> ZRANGEBYSCORE importance -inf inf WITHSCORES
1) "gog"
2) "212"
3) "yt"
4) "1152"
5) "tw"
6) "10156"
```

Подвоїмо значення відсортованої множини **ratings**:

```
127.0.0.1:6379> ZUNIONSTORE ratings 1 ratings WEIGHTS 2
(integer) 3
127.0.0.1:6379> ZRANGE ratings 0 -1 WITHSCORES
1) "gog"
2) "112"
3) "yt"
4) "152"
5) "tw"
6) "156"
127.0.0.1:6379>
```

Зчитування за інтервалом значень

Для зчитування вмісту відсортованої множини за вказаним інтервалом існує команда **ZRANGE** — подібно до **LRANGE**, застосованої до списку. Зчитаємо кілька значень з початку списку **ratings**.

```
127.0.0.1:6379> ZRANGE ratings 0 1
1) "gog"
2) "yt"
```

Для зчитування за зменшенням значень лічильника запит доповнюють префіксом REV, наприклад **ZREVRANGE**. Параметр WITHSCORES позначає потребу зчитування значень лічильника

```
127.0.0.1:6379> ZREVRANGE ratings 0 -1 WITHSCORES
1) "tw"
2) "156"
3) "yt"
4) "152"
5) "gog"
6) "112"
```

Стандартно нижня й верхня межі враховуються для вказаного інтервалу.

```
127.0.0.1:6379> ZRANGEBYSCORE ratings 1 155
1) "gog"
2) "yt"
```

Допускається зчитування значень за інтервалом з нескінченними межами

```
127.0.0.1:6379> ZRANGEBYSCORE ratings -inf inf
1) "gog"
2) "yt"
3) "tw"
```

Команда **ZREVRANGEBYSCORE** зчитує вміст відсортованої множини за зменшенням значення лічильника

```
127.0.0.1:6379> ZREVRANGEBYSCORE ratings inf -inf
1) "tw"
2) "yt"
3) "gog"
```

Для видалення екземплярів даних за інтервалом також реалізовано команди **ZREMRANGEBYRANK** і **ZREMRANGEBYSCORE**.

Транзакції

Команда **MULTI** в Redis позначає початок атомарного блоку команд. Коли в єдиному блоці розміщується кілька команд, наприклад **SET** і **INCR**, тоді або всі виконуються коректно, або всі не виконуються, — частковий результат неможливий. Транзакція починається командою **MULTI** і закінчується командою **EXEC** — виконання списку команд транзакції

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET store http://store.com
QUEUED
127.0.0.1:6379(TX)> INCR count
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) (integer) 1
127.0.0.1:6379>
```

Подібно до транзакцій в реляційних СБД, в блоці MULTI команди в момент зазначення в консолі відразу не виконуються. Оскільки виконання сервера однопотокове, команди спочатку розміщуються в черзі й після команди EXEC виконуються послідовно

Об'єднання команд

В межах запиту допускається об'єднання кількох команд, розподілених «\r\n». Виконуємо запит до redis-cli з консолі ОС.

```
# echo "PING \r\n PING \r\n PING \r\n PING \r\n" | redis-cli
PONG
PONG
PONG
PONG
```

Моделі комунікації

Блокована черга

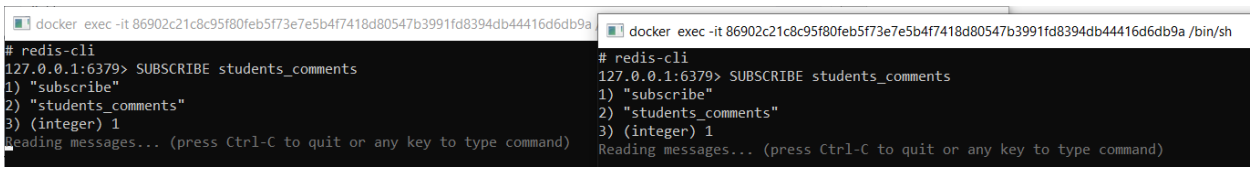
Створимо підсистему транспортування повідомлень, коли різноманітні джерела дозаписують повідомлення в чергу, а читач їх зчитує. Для вирішення такого завдання в Redis реалізовано блоковані черги. Виконуємо запуск додаткової консолі клієнтської утиліти redis-cli. Команда BRPOP очікує протягом вказаного інтервалу часу, зазначеного в секундах, створення екземплярів даних блокованої черги Перемістимося до першої консолі й командою LPUSH дозапишемо повідомлення в блоковану чергу students_comments .

```
127.0.0.1:6379> LPUSH students_comments "KPI is the best university!"
(integer) 1
127.0.0.1:6379> BRPOP students_comments 100
1) "students_comments"
2) "KPI is the best university!"
```

Модель «джерело / читач»

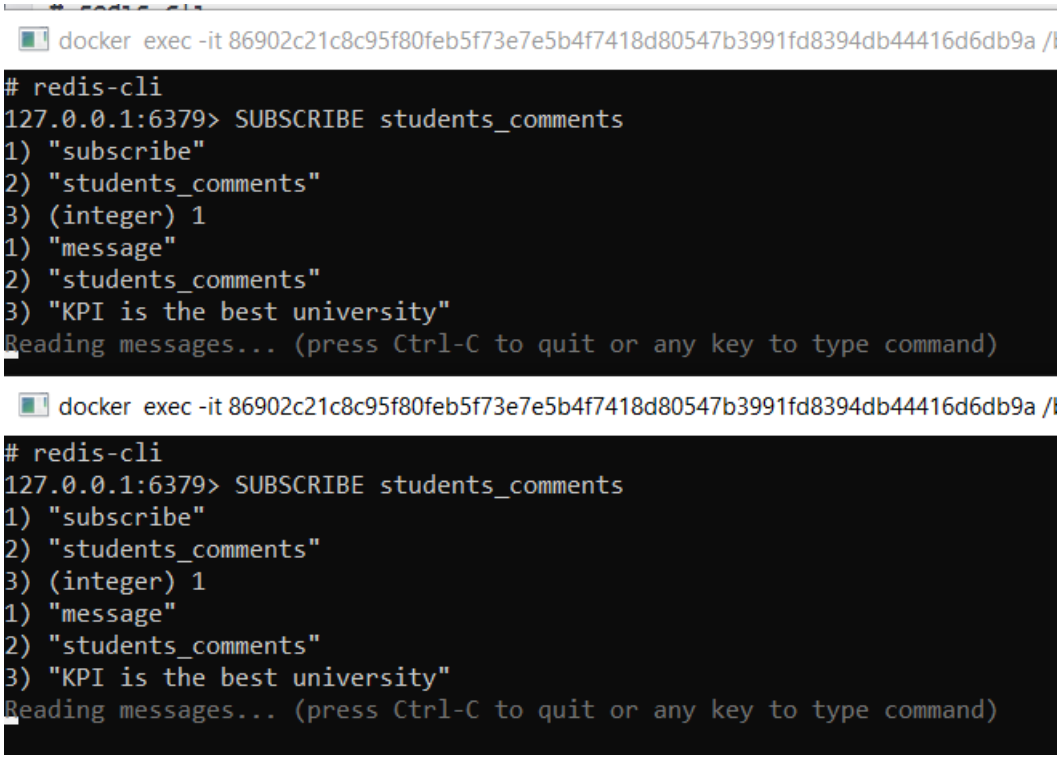
Удосконалимо коментування, вже розроблене на базі блокованих черг, забезпечивши зчитування коментарів клієнтами Виконуємо запуск двох додаткових консолей Redis. Під’єднаємо клієнтів до джерела «коментарів» (students_comments).

Після встановлення з’єднання з джерелом консоль перебуває в стані зчитування:



Сформуємо з джерела payment_comments строкове повідомлення. Команда **PUBLISH** повертає значення 2, яке дорівнює кількості читачів:

```
# redis-cli
127.0.0.1:6379> PUBLISH students_comments "KPI is the best university"
(integer) 2
```



Для припинення виконання сервера в консолі сервера треба перемикнутися клавішами **CTRL+C**

Практичні завдання

1. Встановіть бібліотеку мови програмування на ваш вибір і програмно реалізуйте підключення до сервера Redis. Створіть і збільште на одиницю в межах транзакції цілочисельне значення.

Python:

main.py

```python
import redis
# Підключення до Redis
redis_connection = redis.Redis(host='localhost', port=6379, db=0)
with redis_connection.pipeline() as transaction:
    transaction.set('new_task', 0)  # Створення цілочисельного значення в межах транзакції
    transaction.incr('new_task')  # Збільшення значення на одиницю в межах транзакції
    transaction.execute()
# Зчитування результату
print(redis_connection.get('new_task'))
```

```
C:\Users\user\PycharmProjects\pythonProj
b'1'

Process finished with exit code 0
```

```
# redis-cli
127.0.0.1:6379> GET new_task
"1"
127.0.0.1:6379>
```

2. Застосовуючи встановлену бібліотеку API, програмно реалізуйте: процес № 1 — дозаписує дані в блоковану чергу, процес № 2 — зчитує екземпляри даних блокованої черги й дозаписує їх у текстовий файл.

main.py output_data.txt

```python
# Додавання даних до черги
for i in range(0, 5):
    data_to_push = f"Data:{i}"
    redis_conn.lpush('task_queue', data_to_push)

# Запис отриманих даних у текстовий файл
output_file_path = 'output_data.txt'
with open(output_file_path, 'a') as output_file:
    while True:
        queue_item = redis_conn.blpop('task_queue', timeout=2)  # Отримання даних із блокованої черги
        if queue_item is None: # Перевірка, чи отримано значення
            break
        output_file.write(str(queue_item) + '\n') # Запис отриманих даних у текстовий файл
print("Processing Completed")
```

```
(b'task_queue', b'Data:4')
(b'task_queue', b'Data:3')
(b'task_queue', b'Data:2')
(b'task_queue', b'Data:1')
(b'task_queue', b'Data:0')
```

Висновок

У ході виконання лабораторної роботи №5 були набуті навички роботи зі СБД Redis, також були виконані певні CRUD запити до БД. Були вивчені нові структури даних, такі як: список, стек, хеш.